

USER_GUIDE

HelixCluster — End-User Guide

Revision: 1 **Last modified:** 2026-06-03 **Maintainer:** HelixCluster docs **Status:** active **Audience:** end users who want to submit and run work on a HelixCluster deployment

0. Read this first — what is actually runnable today

HelixCluster is a distributed GPU-sharing / compute-cluster OS. This guide describes the **request/session model and the request flow as the code implements it today**, and it is deliberately honest about the boundary between what you can exercise on a single host and what requires a real, multi-node deployment with GPU providers.

Two scopes are used throughout this document:

- **Single-host runnable today.** You can build and run the gateway and the control-plane services on one machine and drive the real HTTP / gRPC surfaces against them. The handlers, request/response shapes, auth, and the E2EE primitives are all real code (cited per section).
- **Requires a deployed cluster / real GPU providers — not yet validated end-to-end.** The full path *workload* → *client* → *E2EE* → *network* → *miner* → *response* across a real fleet is tracked as the explicit end-to-end validation gate **HXC-1491** (docs/issues.md, Status: Queued, Task | P0). Until HXC-1491 is closed with captured evidence, treat any claim of a fully-working multi-node confidential-inference round trip as **infra-gated / not yet validated end-to-end**. This is stated again at the relevant sections below so it is never implied otherwise.

If a capability is not cited to a real handler/file below, it is not part of this guide.

1. What HelixCluster offers a user

From an end user's point of view the system exposes:

1. **A session model** — you create a *session* that represents an allocation of compute (CPU millicores, memory bytes, and GPU IDs) owned by you, in a

chosen execution mode and backend. Sessions are the unit you create, query, update, and delete.

2. **An interactive terminal stream** — an established session can be attached to over a WebSocket and bridged to a real PTY/shell, so you get an interactive terminal into your allocation.
3. **An internal AI inference route** — you can submit a model + prompt and get a completion back, optionally over a confidential (TEE / end-to-end encrypted) path.
4. **A pool-utilization read** — a simple read of current cluster CPU / memory / GPU utilization percentages.

These are reached through two surfaces:

- The **HTTP gateway** (`cmd/helix-gateway`, package `internal/gateway`) — the front door. It serves a small REST surface directly and reverse-proxies the rest to backend services.
- The backend **gRPC services**, principally the **Session service** (`cmd/helix-session` + `internal/session`, contract in `api/v1/session.proto`) and the **Scheduler service** (`cmd/helix-scheduler`, contract in `api/v1/scheduler.proto`).

2. The request/session model

2.1 What a session is

The canonical session shape is defined in `api/v1/types.proto` (message `Session`) and `api/v1/session.proto` (`SessionService`). A session carries:

Field	Meaning
<code>id</code>	Server-assigned session identifier
<code>name</code>	Your name for the session (required on create)
<code>owner</code>	Owning principal (required on create)
<code>status</code>	Lifecycle status (see below)
<code>mode</code>	Execution mode label, round-tripped via session labels
<code>backend</code>	Terminal backend: <code>tmux</code> , <code>zellij</code> , or <code>screen</code>
<code>node_id</code>	Node the session is placed on
<code>resources</code>	ResourceAllocation: <code>cpu_millicores</code> , <code>memory_bytes</code> , <code>gpu_ids[]</code>

Resource requests are validated server-side: negative `cpu_millicores` or `memory_bytes` are rejected, and an unknown backend is rejected (`internal/session/server.go`, `CreateSession`).

2.2 Session lifecycle / statuses

The Session server recognizes exactly these status names (`internal/session/server.go`, `validStatus`):

pending, creating, running, migrating, paused, terminated, failed.

An unknown status supplied to `ListSessions` (filter) or `UpdateSession` is rejected with `InvalidArgument` rather than silently coerced — this is a deliberate anti-foot-gun: a typo such as `runnig` will **not** quietly reset a running session to pending.

2.3 Session operations (gRPC `SessionService`)

From `api/v1/session.proto`, implemented in `internal/session/server.go`:

- `CreateSession(CreateSessionRequest) → Session` — requires name and owner; optional mode, backend, resources.
- `GetSession(GetSessionRequest) → Session` — by `session_id`.
- `ListSessions(ListSessionsRequest) → ListSessionsResponse` — optional owner and status filters.
- `UpdateSession(UpdateSessionRequest) → Session` — change status and/or resources.
- `DeleteSession>DeleteSessionRequest) → DeleteSessionResponse` — terminates the session.

What backs a session (real process, not a stub): when `tmux` is present on the host the manager uses a real `TmuxBackend`; when it is absent it falls back to a native local-PTY shell backend — never a nil backend that pretends to be running (`internal/session/server.go`, `newRealBackend`). So on a single host the session genuinely launches a real process.

3. How you submit work and get a response — the real API/flow

3.1 The HTTP entry points (real handlers)

The gateway is a plain `http.Handler` (`internal/gateway/gateway.go`, `Gateway.ServeHTTP`). It dispatches in this order:

1. `GET /health → {"status":"healthy"}` (always served by the gateway).
2. **REST surface** (`internal/gateway/api.go`), served directly when present:
 - `POST /v1/sessions`
 - `GET /v1/pool/utilization`
 - `GET /openapi.json`
3. **Inference route** (`internal/gateway/inference.go`):
 - `POST /v1/ai/infer`
4. **Reverse-proxied prefixes** (`internal/gateway/gateway.go`, routes):
 - `/api/v1/scheduler/ → scheduler service (:50052)`
 - `/api/v1/session/ → session service (:50053)`
 - `/api/v1/build/ → build service (:50051)`

◦ `/api/v1/node/` → node service (:50054)

Every proxied response is stamped with the `X-Helix-Gateway: true` header so you can confirm a response actually traversed the gateway. When a backend is unreachable the gateway returns a real machine-readable `502 {"error":"backend unavailable","status":502}` rather than an empty body.

The REST surface is also published as OpenAPI 3.0 at `GET /openapi.json` (embedded spec in `internal/gateway/api.go`; YAML companion `internal/gateway/openapi.yaml`).

3.2 Create a session over REST

`POST /v1/sessions` (`internal/gateway/api.go, handleSessions`). Request body is a `SessionSpec` whose resource is an opaque JSON object forwarded to the backend:

```
POST /v1/sessions
Content-Type: application/json
```

```
{ "resource": { "cpu_millicores": 2000, "memory_bytes":
4294967296, "gpu_ids": ["gpu-0"] } }
```

On success the gateway responds **201 Created** with:

```
{ "id": "session-0001", "status": "CREATING", "spec": {
  "cpu_millicores": 2000, "memory_bytes": 4294967296,
  "gpu_ids": ["gpu-0"] } }
```

Notes grounded in the code:

- The returned status on this REST path is always `CREATING` (`handleSessions`). Richer lifecycle transitions are observed through the gRPC `SessionService` (`GetSession / UpdateSession`), not this REST 201.
- Malformed JSON yields `400` with a JSON error body.
- The default in-process session backend assigns sequential IDs (`session-0001`, ...) when no real backend is injected (`defaultSessionBackend`). A real deployment injects a real `SessionBackend` via `WithSessionBackend`.

3.3 Submit an inference request

`POST /v1/ai/infer` (`internal/gateway/inference.go, handleInfer`). Request:

```
POST /v1/ai/infer
Content-Type: application/json
```

```
{ "model": "some-model", "prompt": "hello", "tee_required":
false }
```

- `model` and `prompt` are both required; missing either yields `400`.
- On success: `200` with `{ "model": "...", "completion": "...", "e2ee_enabled": <bool> }`.

- The handler forwards to an injected `ChutesInferenceClient.Complete(model, prompt, teeRequired)`. **If no inference client is configured, the request honestly fails** — the default client returns a typed `InferenceError("no inference client configured")` and the handler returns 502 with a JSON error body. It never fabricates a completion (defaultInferenceClient, the 502 branch in `handleInfer`). This is by design (anti-bluff): an unconfigured inference backend is surfaced, not faked.

Confidential / TEE path marker. When you set `"tee_required": true`, the request is routed through the client's confidential-compute / E2EE-wrapping path and the response is stamped with `X-E2EE-Enabled: true` (and `e2ee_enabled: true` in the body) so you have sink-side proof the confidential path was taken (inference.go, the `if req.TEERequired` branch). When false, the marker is absent.

Scope: the gateway-side routing, the TEE flag, the marker, and the typed-error behavior are real and single-host-exercisable with an injected client. A real model completion requires a configured `ChutesInferenceClient` talking to a real provider — that, and the full multi-node round trip, are **infra-gated / not yet validated end-to-end (HXC-1491)**. See §5.

3.4 Read pool utilization

`GET /v1/pool/utilization` (internal/gateway/api.go, `handlePoolUtilization`) returns `{ "cpu_pct", "mem_pct", "gpu_pct" }`. With no real `PoolBackend` injected the default backend returns zeros (a value of 0.0 means “not measured / unavailable”, per the type doc in `api.go`), so on a bare single host expect zeros until a real backend is wired.

3.5 Attach an interactive terminal to a session

The Session service also exposes a WebSocket terminal bridge at `/stream?session=<id>` (`cmd/helix-session/stream.go`, `StreamSessionHandler`). It upgrades the connection, starts a real PTY/shell (`realPTYProvider`, using your `$SHELL` or `/bin/sh`), and bridges it with length-typed envelopes (`pkg/websocket`):

- client → PTY: Input envelopes (stdin)
- PTY → client: Output envelopes (stdout/stderr)
- client → PTY: Resize envelopes (4-byte big-endian rows,cols; build with `ResizePayload`)
- Heartbeat envelopes are echoed back

This is a real terminal into a real process on the host running the session service.

3.6 The scheduler (placement) contract

Placement is the Scheduler service (`api/v1/scheduler.proto`, served by `cmd/helix-scheduler`). It is reached through the gateway prefix `/api/v1/scheduler/`. The gRPC contract:

- `ScheduleJob(ScheduleJobRequest) → ScheduleJobResponse` — request carries `job_id`, `session_id`, requirements (`ResourceAllocation`), free-

form constraints, and tier/power controls (`allowed_tiers`, `min_tier`, `power_budget_watts`). Response: `job_id`, `node_id`, `scheduled`.

- `CancelJob`, `GetJobStatus`, `ListJobs`.
- `StreamJobEvents(StreamJobEventsRequest)` → `stream JobEvent` — a server stream of `{job_id, event_type, message, timestamp}` for live job progress.

Scope: the scheduler contract and service are real. Meaningful placement decisions require registered nodes / a real fleet; on a single host you can drive the RPCs but placement targets are limited to what is registered.

4. Authentication and authorization

Auth is **off by default** and turned on by deploying the gateway with an `AuthPolicy` via `WithAuth` (`internal/gateway/auth.go`, `internal/gateway/gateway.go`). When enabled:

- You must present a bearer token: `Authorization: Bearer <jwt>`. The token is validated for structure, **HS256** signature, and expiry (`auth.go`, `authorize`, via `pkg/jwt.ParseToken`).
- Authorization is **scope-based RBAC** with longest-prefix matching: each path prefix maps to a required scope; a `DefaultScope` applies when no prefix matches; an empty required scope means “any valid token” (`AuthPolicy.RequiredScope`). Scopes are read from the token’s `scope / scopes / roles` claims (`claimScopes`).
- Rejections are explicit: `401` for a missing/invalid/expired token, `403` for insufficient scope, each with a JSON body and an `X-Helix-Deny-Reason` header for diagnosis (`writeAuthError`). The upstream is never contacted on a rejected request.

The REST surface (`/v1/sessions`, `/v1/pool/...`, `/openapi.json`) is dispatched **before** the proxy auth gate and applies its own validation; the JWT/RBAC gate guards the reverse-proxied `/api/v1/...` prefixes.

5. Confidential inference / E2EE (high level)

HelixCluster ships a real post-quantum end-to-end encryption package, `security/pkg/e2ee`, used to make inference confidential between a client and a worker. At a high level (see the package docs in `security/pkg/e2ee/package.go`):

- **Key establishment** uses **ML-KEM-768** (NIST FIPS 203, via Go stdlib `crypto/mlkem`). An initiator publishes an ephemeral encapsulation key; the responder encapsulates against it; both derive the same shared secret.
- **Session key derivation** uses **HKDF-SHA-256**, with the salt bound to the exact KEM ciphertext (`deriveSessionKey`).
- **Record protection** is AEAD — **AES-256-GCM** by default, with **ChaCha20-Poly1305** selectable (`SessionConfig.Suite`). Each record uses a unique nonce and the session enforces single-use on `Open`, so replays are rejected (`ErrNonceReuse`) and tampering is rejected (`ErrOpen`).

- **Transport framing** is a length-prefixed record stream over any `io.ReadWriter` (`security/pkg/e2ee/transport.go`, `Transport`).
- **Confidential streaming** (`security/pkg/e2ee/sse.go`, `StreamDecryptor`) delivers a completion as an `e2e_init` handshake frame followed by an ordered sequence of independently-sealed SSE chunks; counter-nonce ordering makes the stream tamper- and replay-evident.
- **Per-request response binding** (`security/pkg/e2ee/response_keypair.go`, `ResponseKeypair`): each request carries a fresh ephemeral response public key, so a worker's response is cryptographically openable **only** by the client that issued *that* request — a response sealed for request 1 cannot be opened with request 2's key.

How this connects to the user-facing flow: setting `tee_required: true` on `POST /v1/ai/infer` selects the confidential path and you get the `X-E2EE-Enabled: true` sink-side marker back (§3.3).

Scope / honesty. The e2ee cryptographic primitives, transport, streaming decryptor, and per-request keypair binding are real, tested code in `security/pkg/e2ee`. The gateway TEE flag and marker are real. What is **not yet validated end-to-end** is the *complete* live path across a deployed fleet — workload → client → E2EE → network → miner → response — which is exactly the scope of **HXC-1491** (`docs/issues.md`, Status: Queued). Until that gate closes with captured evidence, treat full multi-node confidential inference as **infra-gated, not yet proven end-to-end**. Related roadmap gates also note this: see `docs/PHASE_8C_EXIT_GATE_EVIDENCE.md` (G16/G17), which point at HXC-1491 as the live-flow ticket.

6. Running the front door on a single host (today)

The gateway binary is configured entirely from the environment (`cmd/helix-gateway/main.go`):

- `HELIX_GATEWAY_PORT` — listen port (default 8080; must be a valid integer in 1–65535 or startup fails fast).
- `HELIX_GATEWAY_HOST` — bind host (default all interfaces).

It also mounts Prometheus `/metrics` alongside the gateway handler. The session service is configured by `HELIX_SESSION_HOST` / `HELIX_SESSION_PORT` (default 50053, `cmd/helix-session/main.go`); the scheduler service has the analogous config (`cmd/helix-scheduler/main.go`, default :50052). Each service registers a gRPC health check and shuts down gracefully on `SIGINT`/`SIGTERM`.

On a single host you can:

- Start the gateway and hit `GET /health`, `POST /v1/sessions`, `GET /v1/pool/utilization`, `GET /openapi.json`, and `POST /v1/ai/infer` (the last returns a 502 typed error unless an inference client is wired).
- Start `helix-session` and create/get/list/update/delete sessions over gRPC, and attach a terminal over `/stream`.
- Start `helix-scheduler` and drive the scheduler RPCs.

What you **cannot** prove on a single bare host: real GPU placement across many nodes, real model completions without a configured provider client, and the full confidential-inference round trip — all of which are the deployed-cluster / HXC-1491 scope described above.

7. Quick reference — endpoints and their source

Endpoint	Method	Handler (source)	Scope
/health	GET	internal/gateway/gateway.go	single-host
/v1/sessions	POST	internal/gateway/api.go handleSessions	single-host
/v1/pool/utilization	GET	internal/gateway/api.go handlePoolUtilization	single-host (zeros without real backend)
/openapi.json	GET	internal/gateway/api.go handleOpenAPIJSON	single-host
/v1/ai/infer	POST	internal/gateway/inference.go handleInfer	single-host routing; real completion + full round trip infra-gated (HXC-1491)
/stream?session=<id>	WS	cmd/helix-session/stream.go StreamSessionHandler	single-host (real PTY)
/api/v1/scheduler/*	proxied	internal/gateway/gateway.go → api/v1/scheduler.proto	needs scheduler service / fleet
/api/v1/session/*	proxied	internal/gateway/gateway.go → api/v1/session.proto	needs session service
/api/v1/build/*, /api/v1/node/*	proxied	internal/gateway/gateway.go	needs respective services

Sources verified

All claims above are grounded in the repository at the documented revision date:

- internal/gateway/gateway.go, internal/gateway/api.go, internal/gateway/auth.go, internal/gateway/inference.go, internal/gateway/openapi.yaml
- cmd/helix-gateway/main.go, cmd/helix-session/main.go, cmd/helix-session/stream.go, cmd/helix-scheduler/main.go

- `internal/session/server.go`
- `api/v1/session.proto`, `api/v1/scheduler.proto`, `api/v1/types.proto`
- `security/pkg/e2ee/package.go`, `security/pkg/e2ee/transport.go`,
`security/pkg/e2ee/sse.go`, `security/pkg/e2ee/response_keypair.go`
- `docs/issues.md` (HXC-1491), `docs/PHASE_8C_EXIT_GATE_EVIDENCE.md`
(G16/G17)